

Міністерство освіти і науки України
Львівський національний університет імені Івана Франка
Факультет електроніки та комп'ютерних технологій

ЗВІТ

про виконання лабораторної роботи № 7
з курсу “Операційні системи реального часу”
на тему: “Обробка даних із сенсорів у багатозадачній системі”

Виконав:
Студент. гр. ФеП-41 Фіняк Олег
Перевірив:
Павлик М.Р.

Львів – 2025

Мета роботи. Навчитися організовувати взаємодію між задачами у системі реального часу, що одночасно працюють із кількома сенсорами.

```
/* Includes */
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

/* Pico SDK */
#include "pico/stdlib.h"

/* FreeRTOS */
#include "FreeRTOS.h"
#include "task.h"
#include "queue.h"
#include "semphr.h"

/* Waveshare Libs (OLED) */
#include "OLED_1in5.h"
#include "GUI_Paint.h"
#include "DEV_Config.h"

/* Sensor Libs */
#include "SHTC3.h"
#include "SGP40.h"
#include "QMI8658.h"

/* Defines */
#define BUTTON_PIN 3

/* RTOS Handles */
static SemaphoreHandle_t i2c_mutex;
static QueueHandle_t xQueueButton;
static QueueHandle_t xQueueSHTC3;
static QueueHandle_t xQueueSGP40;
static QueueHandle_t xQueueQMI8658;

/* Global OLED Buffer */
static UBYTE *BlackImage;

/* Data Structures for Queues */
typedef struct {
    float temp;
    float hum;
} SHTC3_data_t;

typedef struct {
    float acc[3];
    float gyro[3];
} QMI8658_data_t;

/**
 * @brief Задача опитування кнопки з debounce.
 */
void vButtonTask(void *pvParameters) {
    gpio_init(BUTTON_PIN);
    gpio_set_dir(BUTTON_PIN, GPIO_IN);
    gpio_pull_up(BUTTON_PIN);

    bool last_state = true; // true = відпущена (через pull-up)

    while (1) {
```

```

bool current_state = gpio_get(BUTTON_PIN);

// Якщо кнопка натиснута (false) і до цього була відпущена (true)
if (current_state == false && last_state == true) {
    // Debounce delay
    vTaskDelay(pdMS_TO_TICKS(50));
    current_state = gpio_get(BUTTON_PIN);

    // Якщо все ще натиснута, відправляємо подію
    if (current_state == false) {
        bool pressed = true;
        xQueueSend(xQueueButton, &pressed, 0);
    }
}

last_state = current_state;
vTaskDelay(pdMS_TO_TICKS(50)); // Опитування кожні 50 мс
}
}

/**
 * @brief Задача зчитування сенсора температури/вологості SHTC3.
 */
void vSHTC3Task(void *pvParameters) {
    SHTC3_data_t data;

    while (1) {
        // Захоплюємо м'ютекс для доступу до I2C
        if (xSemaphoreTake(i2c_mutex, portMAX_DELAY) == pdTRUE) {

            SHTC3_Measurement(&data.temp, &data.hum);

            // Відпускаємо м'ютекс
            xSemaphoreGive(i2c_mutex);

            // Відправляємо дані в чергу
            xQueueSend(xQueueSHTC3, &data, 0);
        }

        vTaskDelay(pdMS_TO_TICKS(100)); // Опитування кожні 100 мс
    }
}

/**
 * @brief Задача зчитування сенсора якості повітря SGP40.
 */
void vSGP40Task(void *pvParameters) {
    uint32_t voc_index;

    while (1) {
        if (xSemaphoreTake(i2c_mutex, portMAX_DELAY) == pdTRUE) {

            // Використовуємо T=25C, H=50% як у прикладі
            voc_index = SGP40_MeasureVOC(25, 50);

            xSemaphoreGive(i2c_mutex);

            xQueueSend(xQueueSGP40, &voc_index, 0);
        }

        vTaskDelay(pdMS_TO_TICKS(100)); // Опитування кожні 100 мс
    }
}

```

```

/**
 * @brief Задача зчитування акселерометра/гіроскопа QMI8658.
 *
 */
void vQMI8658Task(void *pvParameters) {
    QMI8658_data_t data;
    unsigned int tim_count = 0; // Як у прикладі

    while (1) {
        if (xSemaphoreTake(i2c_mutex, portMAX_DELAY) == pdTRUE) {

            QMI8658_read_xyz(data.acc, data.gyro, &tim_count);

            xSemaphoreGive(i2c_mutex);

            xQueueSend(xQueueQMI8658, &data, 0);
        }

        vTaskDelay(pdMS_TO_TICKS(100)); // Опитування кожні 100 мс
    }
}

/**
 * @brief Задача відображення даних на OLED-дисплеї.
 *
 */
void vOLEDTask(void *pvParameters) {
    static bool display_on = true;

    // Локальні змінні для зберігання останніх даних
    SHTC3_data_t shtc3_data = {0};
    uint32_t voc_data = 0;
    QMI8658_data_t qmi_data = {{0}, {0}};
    bool button_pressed;
    char str[30];

    while (1) {
        // 1. Перевірка натискання кнопки (неблокуюча)
        if (xQueueReceive(xQueueButton, &button_pressed, 0) == pdPASS) {
            if (button_pressed) {
                display_on = !display_on;
            }
        }

        // 2. Отримання даних з черг (неблокуюче)
        xQueueReceive(xQueueSHTC3, &shtc3_data, 0);
        xQueueReceive(xQueueSGP40, &voc_data, 0);
        xQueueReceive(xQueueQMI8658, &qmi_data, 0);

        // 3. Відображення
        Paint_SelectImage(BlackImage);
        Paint_Clear(BLACK);

        if (display_on) {
            // SHTC3
            sprintf(str, "T: %.2f C", shtc3_data.temp);
            Paint_DrawString_EN(5, 10, str, &Font12, WHITE, BLACK);
            sprintf(str, "H: %.2f %%", shtc3_data.hum);
            Paint_DrawString_EN(5, 25, str, &Font12, WHITE, BLACK);

            // SGP40
            sprintf(str, "VOC: %lu", voc_data);
            Paint_DrawString_EN(5, 40, str, &Font12, WHITE, BLACK);
        }
    }
}

```

```

    // QMI8658 (тільки Accel для економії місця)
    sprintf(str, "AccX: %.2f", qmi_data.acc[0]);
    Paint_DrawString_EN(5, 55, str, &Font12, WHITE, BLACK);
    sprintf(str, "AccY: %.2f", qmi_data.acc[1]);
    Paint_DrawString_EN(5, 70, str, &Font12, WHITE, BLACK);
    sprintf(str, "AccZ: %.2f", qmi_data.acc[2]);
    Paint_DrawString_EN(5, 85, str, &Font12, WHITE, BLACK);

} else {
    // Дисплей вимкнено
    Paint_DrawString_EN(20, 50, "Display OFF", &Font12, WHITE, BLACK);
}

OLED_1in5_Display(BlackImage);

// Оновлення екрану кожні 200-300 мс
vTaskDelay(pdMS_TO_TICKS(200));
}
}

int main(void) {
    // Ініціалізація системи
    stdio_init_all();
    if (DEV_Module_Init() != 0) {
        printf("DEV_Module_Init failed\r\n");
        while(1);
    }
    printf("DEV_Module_Init OK\r\n");

    // Ініціалізація OLED
    OLED_1in5_Init();
    OLED_1in5_Clear(0);

    // Створення буфера зображення
    UWORD Imagesize = ((OLED_1in5_WIDTH % 2 == 0) ? (OLED_1in5_WIDTH / 2) : (OLED_1in5_WIDTH / 2 +
1)) * OLED_1in5_HEIGHT;

    if ((BlackImage = (UBYTE *)malloc(Imagesize)) == NULL) {
        printf("Failed to apply for black memory...\r\n");
        while(1);
    }
    Paint_NewImage(BlackImage, OLED_1in5_WIDTH, OLED_1in5_HEIGHT, 0, WHITE);
    Paint_SetScale(16);
    Paint_Clear(BLACK);
    OLED_1in5_Display(BlackImage);

    // Ініціалізація сенсорів (повинна бути до запуску задач!)
    // Використовуємо м'ютекс навіть для ініціалізації про всяк випадок
    i2c_mutex = xSemaphoreCreateMutex();

    xSemaphoreTake(i2c_mutex, portMAX_DELAY);
    SHTC3_Init();
    SGP40_init();
    QMI8658_init();
    xSemaphoreGive(i2c_mutex);
    printf("Sensors Init OK\r\n");

    // Створення черг
    xQueueButton = xQueueCreate(1, sizeof(bool));
    xQueueSHTC3 = xQueueCreate(1, sizeof(SHTC3_data_t));
    xQueueSGP40 = xQueueCreate(1, sizeof(uint32_t));
    xQueueQMI8658 = xQueueCreate(1, sizeof(QMI8658_data_t));

    // Створення задач

```

```
xTaskCreate(vButtonTask, "ButtonTask", 256, NULL, 1, NULL);
xTaskCreate(vSHTC3Task, "SHTC3Task", 256, NULL, 1, NULL);
xTaskCreate(vSGP40Task, "SGP40Task", 256, NULL, 1, NULL);
xTaskCreate(vQMI8658Task, "QMI8658Task", 256, NULL, 1, NULL);
xTaskCreate(vOLEDTask, "OLEDTask", 512, NULL, 2, NULL); // OLED вимагає більше стеку
```

```
// Запуск планувальника
vTaskStartScheduler();
```

```
// Цей код не повинен виконуватися
while (1) {
    // NOP
}
```

Висновок: У ході роботи створено багатозадачну систему FreeRTOS, у якій кілька сенсорних задач паралельно зчитують дані, використовуючи м'ютекс для коректного доступу до спільної I²C шини. Передавання інформації між задачами реалізовано через черги, що забезпечує повністю безпечний обмін даними. Задача дисплея агрегує результати та оновлює OLED з визначеною періодичністю. Додано керування живленням дисплея кнопкою, що дозволяє перемикати стан без переривання роботи сенсорів.